1. Introduction

1.1 Overview

The housing market is highly dynamic, with prices influenced by numerous factors, making accurate price estimation a valuable tool for buyers, sellers, and investors. This project aims to develop a machine learning-based house price prediction model tailored to the Nepali housing market. Leveraging a dataset scraped from actual Nepali real estate listings (which was found in Kaggle), the model will analyse various property features to provide reliable price estimates. The target variable is the property price, and independent variables include features like location, land area, number of floors, and road access.

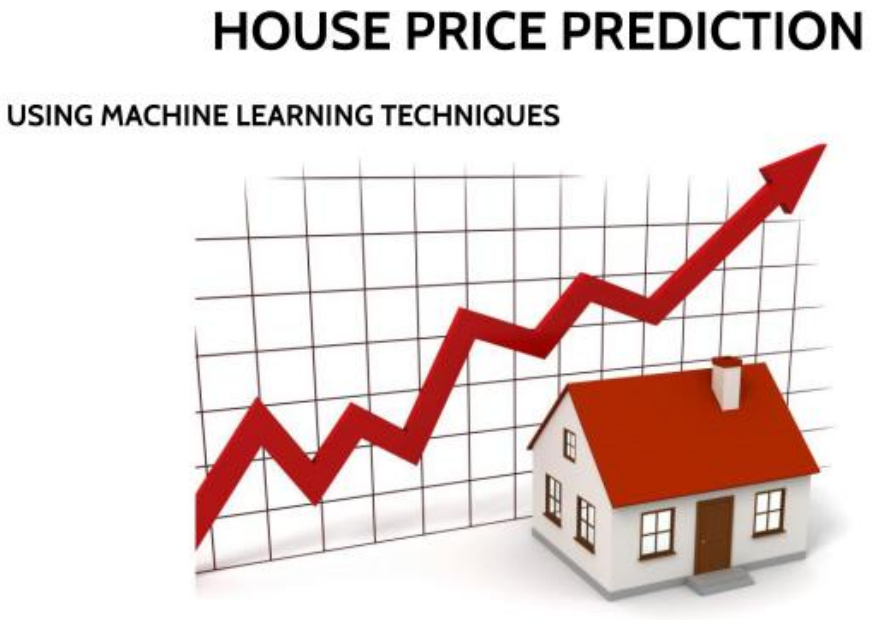


Figure 1:house price prediction

The implementation is carried out using Python, employing machine learning algorithms such as Linear Regression and Artificial Neural Networks (ANN) as primary models. Ensemble learning techniques are considered future plan for further enhanced performance.

1.2 Machine Learning Concepts Applied

Machine Learning algorithms can enable systems to make numbers backed prediction. House price prediction falls under regression problem of Machine Learning. For this problem we can use ML models from different families like linear regression from linear model, DecisionTreeRegressor from tree-based model, RandomForrestRegressor from ensemble learning, XGBoostRegressor from gradient boosting, ANN from neural network. The following machine Learning approaches are used in this project:

- **Linear Regression**:

  Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable (linear-regression, n.d.).

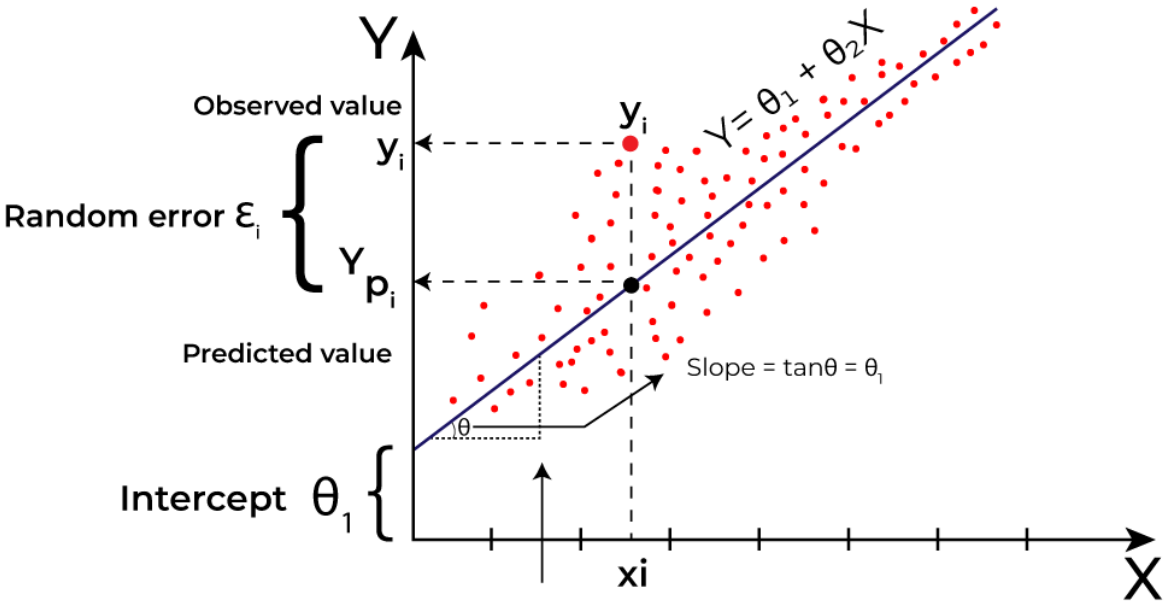


Figure 2: Linear regression

- **Artificial Neural Networks (ANN):**

  Random forest is a commonly used machine learning algorithm, trademarked by Leo Breiman and Adele Cutler, that combines the output of multiple decision trees to reach a single result. Its ease of use and flexibility have fuelled its adoption, as it handles both classification and regression problems (random-forest, n.d.).

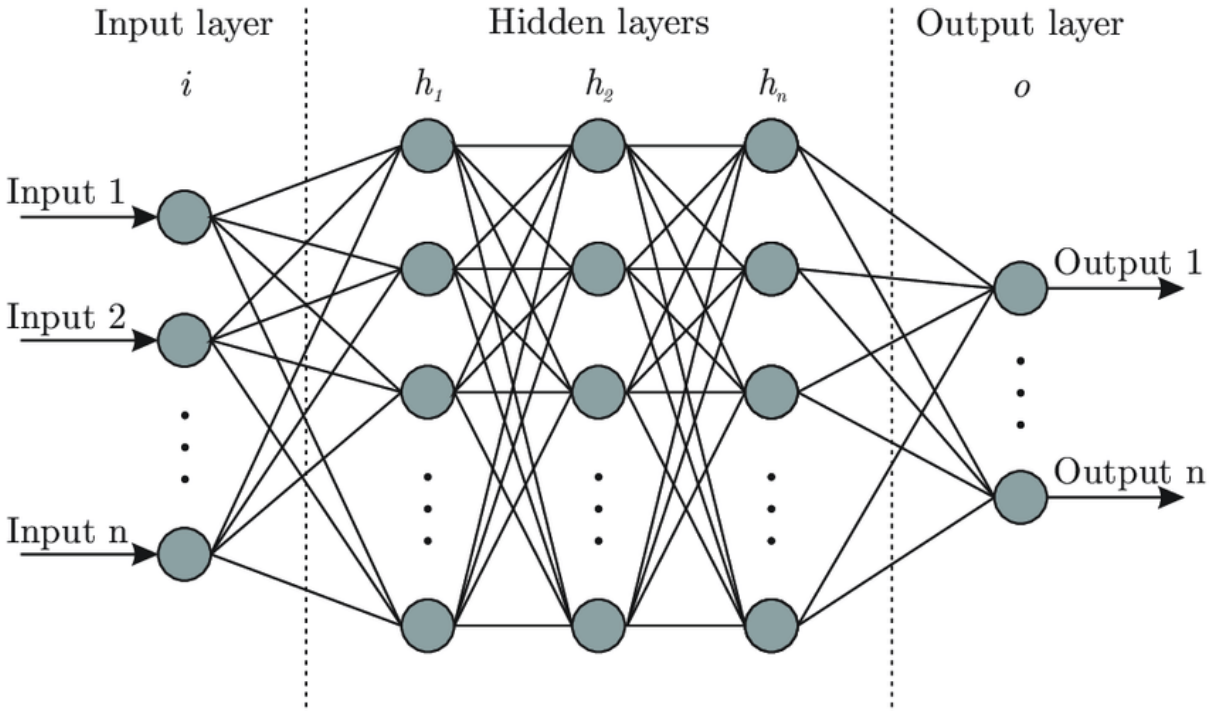


Figure 3: Artificial Neural Network

2. Problem Domain

2.1 Understanding the Problem

Predicting house prices involves analysing tangible factors such as land area, build-up area, number of floors, and road access, as well as intangible aspects like location and amenities. The complex interplay of these factors makes accurate prediction challenging.

The goal is to assist stakeholders in the Nepali real estate market by providing a tool to estimate house prices based on current market conditions and property attributes.

2.2 Dataset Description

The dataset, sourced from Kaggle, contains features such as:

- **TITLE:** Descriptive name of the property
- **LOCATION:** Geographical location
- **PRICE:** Target variable (house price)
- **LAND AREA:** Total land size
- **BUILDUP AREA:** Covered area of the property
- **ROAD ACCESS:** Accessibility in terms of road size
- **FACING:** Direction the property faces
- **FLOOR:** Number of floors
- **BEDROOM:** Number of bedrooms
- **BATHROOM:** Number of bathrooms
- **BUILT YEAR:** Construction year
- **PARKING:** Parking availability
- **AMENITIES:** Additional features like gardens or swimming pools.

	TITLE	LOCATION	PRICE	LAND AREA	BUILDUP AREA	ROAD ACCESS	FACING	FLOOR	BEDROOM	BATHROOM	BUILT YEAR	PARKING	AMENITIES
0	House for Sale	Imadol, Lalitpur	Rs. 2.9 Cr	4.0 aana	NaN	12 Feet	West	3.0	5.0	4.0	2076 B.S	1 CaRs. & 2 Bikes	['Earthquake Resistant', 'Marbel', 'Parquet', ...
1	House for Sale	Satdobato, Lalitpur	Rs. 4.75 Cr	3.0 aana	NaN	10 Feet	West	4.5	5.0	6.0	2076 B.S	2 CaRs. & 2 Bikes	['Earthquake Resistant', 'Parquet', 'Drinking ...
2	4 BHK House for Sale	Imadol, Lalitpur	Rs. 1.99 Cr	2.3 aana	NaN	10 Feet	West	2.5	4.0	4.0	2060 B.S	1 CaRs. & 3 Bikes	['Earthquake Resistant', 'Marbel', 'Parquet', ...
3	Bungalow House for Sale	Bhaisepati, Lalitpur	Rs. 4 Cr	7.0 aana	NaN	12 Feet	North-West	2.5	4.0	3.0	2059 B.S	4 CaRs. & 4 Bikes	['Earthquake Resistant', 'Marbel', 'Parquet', ...
4	House for Rent	Maharajgunj, Kathmandu	Rs. 12000000	6.0 aana	NaN	20 Feet	South	2.0	4.0	4.0	2071 B.S	4 CaRs. & 5 Bikes	['Earthquake Resistant', 'Parquet', 'Parking',...

Figure 4: head of DataFrame of the dataset

3. Solution

3.1 Comprehensive Explanation of the Solution

The proposed solution involves creating a machine learning model tailored to predict house prices in the Nepali market based on Nepali housing market. Key aspects of the solution include:

1. **Problem Understanding:**

Identifying factors influencing house prices and defining the target and feature variables.

2. **Model Selection:**
    - **Linear Regression:** For its simplicity and interpretability in identifying relationships between features and house prices.
    - **Artificial Neural Networks (ANN):** To capture non-linear relationships and complex patterns in the data.
    - **Ensemble Learning:** To combine the strengths of multiple models and improve prediction accuracy.
3. **Feature Engineering:**
    - Selecting, manipulating and transforming simple raw features into appropriate feature
    - Identifying the most impactful features using statistical methods.

4. **Model Evaluation:**

- Assessing performance using metrics like Mean Square Error (MSE), Mean Absolute Error (MAE) Root Mean Square Error (RMSE) and R2score

3.2 Diagrammatical Representation

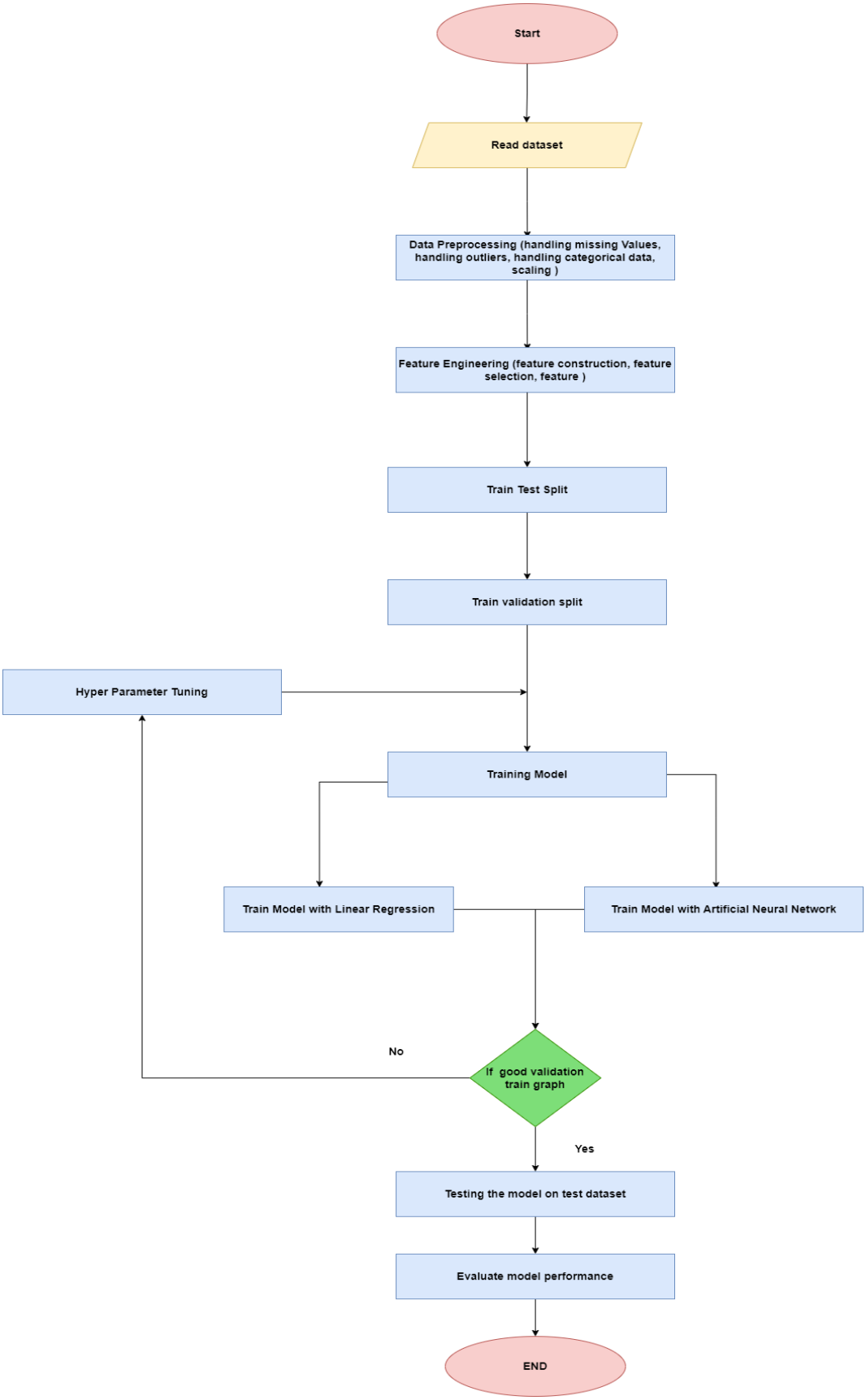


Figure 5: Flowchart of the system

3.3 Explanation of the Development Process

The development process highlights the practical implementation steps taken to create the house price prediction model. It also emphasizes the tools, technologies, and libraries used throughout the project.

i. Data Collection

Data was collected from Kaggle, where it was scraped from real Nepali real estate listings. The dataset contains structured information about various property attributes.

ii. Data Preprocessing

- Handling missing values by removing or imputing them.
- Encoding categorical variables such as **FACING** using **One-Hot Encoding**.
- Scaling numerical features like **PRICE**, **LAND AREA**, and **BUILDUP AREA** using MinMaxScaler or StandardScaler.

iii. Model Training

- Splitting the dataset into training (80%) and testing (20%) subsets using train_test_split.
- Training the following models:
    - **Linear Regression:** For establishing a baseline.
    - **Artificial Neural Networks (ANN):** To capture non-linear patterns.
    - **Ensemble Learning:** Techniques such as Random Forest or Gradient Boosting for improved accuracy.
- Hyperparameter tuning using **GridSearchCV**.

iv. Testing and Evaluation

- Testing the trained models on the test dataset.
- Evaluating models using performance metrics:
  - **MAE (Mean Absolute Error):** Measures the average prediction error.
  - **MSE (Mean Square Error):** Measures the square of prediction error.
  - **RMSE (Root Mean Square Error):** Penalizes larger errors more heavily.
  - **R2-score:** Measure how good the regression line approximates the actual data.
- Comparing results to select the best-performing model.
- Performing Inferencing on user provided data

3.4 Tools, technologies, and libraries employed

1. Programming Language: Python

Python is ideal and widely used language for machine learning and data analysis due to its libraries, community and ease of use.

2. Data Manipulation and Analysis: Pandas, Numpy

- **Pandas:** A powerful library for data manipulation, providing tools for cleaning, analysing, and transforming structured data.



Figure 6: pandas logo

- **Numpy:** A library for numerical computing, for numerical computing which offers support for multi-dimensional arrays and wide range of mathematical operations.



Figure 7: Numpy logo

3. Machine Learning Models and Evaluation: Scikit-learn, TensorFlow/Keras

- **Scikit-learn:** A comprehensive library for machine learning, including tools for model training, evaluation, and preprocessing.



Figure 8: scikit learn logo

- **TensorFlow/Keras:** Frameworks for building and training deep learning models, with user-friendly APIs and support for neural networks.



Figure 9: TensorFlow logo

4. **Data Visualization:** Matplotlib, Seaborn
   - **Matplotlib:** A foundational library for creating static, interactive, and publication-quality plots and graphs.



Figure 10: matplotlib logo

- **Seaborn:** Built on Matplotlib, it provides high-level functions for creating aesthetically pleasing and informative statistical visualizations.



Figure 11: Seaborn logo

5. **Development Environment: VS Code**
   - **Jupyter Notebook:** An interactive environment for writing, running, and visualizing code, widely used for data analysis and prototyping.
   - **Python IDEs:** VS Code provide advanced features such as debugging, syntax highlighting, and project management for efficient development.

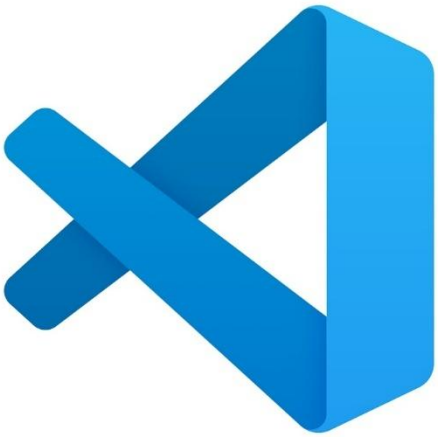


Figure 12: Vs code logo

4. Results

In this project, I have implemented two machine learning algorithms. Linear regression is the ground level algorithm of machine learning, and as many other data scientists, I have first used a linear regression model so that I can compare the linear regression model with other complex level algorithms such ANN. Following are the workarounds of the algorithms implemented in steps:

4.1 Data Loading

4.1.1 Importing Libraries

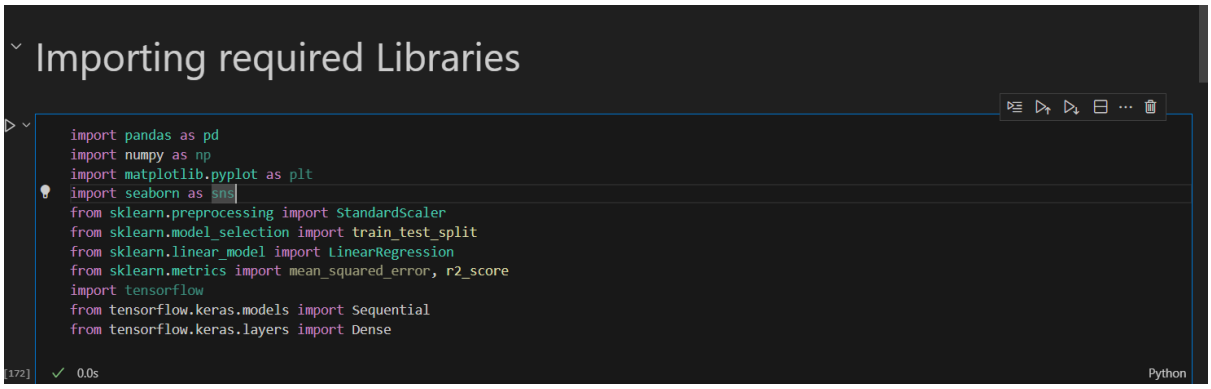


Figure 13: Importing Libraries

4.1.2 Importing datasets

Finding and collecting data for your AI models is the first step and one of the important steps in building a machine learning model.



Figure 14: importing dataset

```
df.head()
```

Python

	TITLE	LOCATION	PRICE	LAND AREA	BUILDUP AREA	ROAD ACCESS	FACING	FLOOR	BEDROOM	BATHROOM	BUILT YEAR	PARKING	AMENITIES
0	House for Sale	Imadol, Lalitpur	Rs. 2.9 Cr	4.0 aana	NaN	12 Feet	West	3.0	5.0	4.0	2076 B.S	1 CaRs. & 2 Bikes	['Earthquake Resistant', 'Marbel', 'Parquet', ...
1	House for Sale	Satdobato, Lalitpur	Rs. 4.75 Cr	3.0 aana	NaN	10 Feet	West	4.5	5.0	6.0	2076 B.S	2 CaRs. & 2 Bikes	['Earthquake Resistant', 'Parquet', 'Drinking ...
2	4 BHK House for Sale	Imadol, Lalitpur	Rs. 1.99 Cr	2.3 aana	NaN	10 Feet	West	2.5	4.0	4.0	2060 B.S	1 CaRs. & 3 Bikes	['Earthquake Resistant', 'Marbel', 'Parquet', ...
3	Bungalow House for Sale	Bhaisepati, Lalitpur	Rs. 4 Cr	7.0 aana	NaN	12 Feet	North-West	2.5	4.0	3.0	2059 B.S	4 CaRs. & 4 Bikes	['Earthquake Resistant', 'Marbel', 'Parquet', ...
4	House for Rent	Maharajgunj, Kathmandu	Rs. 12000000	6.0 aana	NaN	20 Feet	South	2.0	4.0	4.0	2071 B.S	4 CaRs. & 5 Bikes	['Earthquake Resistant', 'Parquet', 'Parking',...

Figure 15: head of DataFrame

4.2 Data Preprocessing and Cleaning

4.2.1 Data Analysis

Analysing the rows and columns of our DataFrame seeing data types, null values, blank spaces.

```
df.isnull().sum()
```

```
TITLE            0
LOCATION         0
PRICE            0
LAND AREA       89
BUILDUP AREA  2699
ROAD ACCESS      9
FACING         206
FLOOR           95
BEDROOM        282
BATHROOM       346
BUILT YEAR      61
PARKING       2786
AMENITIES        0
dtype: int64
```

Figure 16: checking for null values

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3418 entries, 0 to 3417
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   TITLE         3418 non-null   object 
 1   LOCATION      3418 non-null   object 
 2   PRICE         3418 non-null   object 
 3   LAND AREA     3329 non-null   object 
 4   BUILDUP AREA  719 non-null    object 
 5   ROAD ACCESS   3409 non-null   object 
 6   FACING        3212 non-null   object 
 7   FLOOR         3323 non-null   float64
 8   BEDROOM       3136 non-null   float64
 9   BATHROOM      3072 non-null   float64
 10  BUILT YEAR    3357 non-null   object 
 11  PARKING       632 non-null    object 
 12  AMENITIES     3418 non-null   object 
dtypes: float64(3), object(10)
memory usage: 347.3+ KB
```

Figure 17:checking for column, datatype and number of rows

```
df.duplicated().sum()
```

```
0
```

Figure 18:checking for duplicate rows

4.2.2 Preprocessing

Preprocessing is a very important step in machine learning for the preparation of our data before feeding them into our models. How the model will behave is solely dependent upon the data we feed in the model. After data analysis, preprocessing involves imputing null and blank values, checking outliers, normalization, and standardization.

```
# Filter numeric values from LAND AREA (excluding NaN and non-numeric entries)
numeric_land_area = pd.to_numeric(df['LAND AREA'], errors='coerce')

# Plot the distribution of numeric LAND AREA
plt.figure(figsize=(10, 6))
sns.histplot(numeric_land_area.dropna(), kde=True, bins=20, color='blue')
plt.title('Distribution of LAND AREA (Numeric Values)', fontsize=16)
plt.xlabel('LAND AREA', fontsize=14)
plt.ylabel('Frequency', fontsize=14)
plt.grid(True)
plt.show()
```

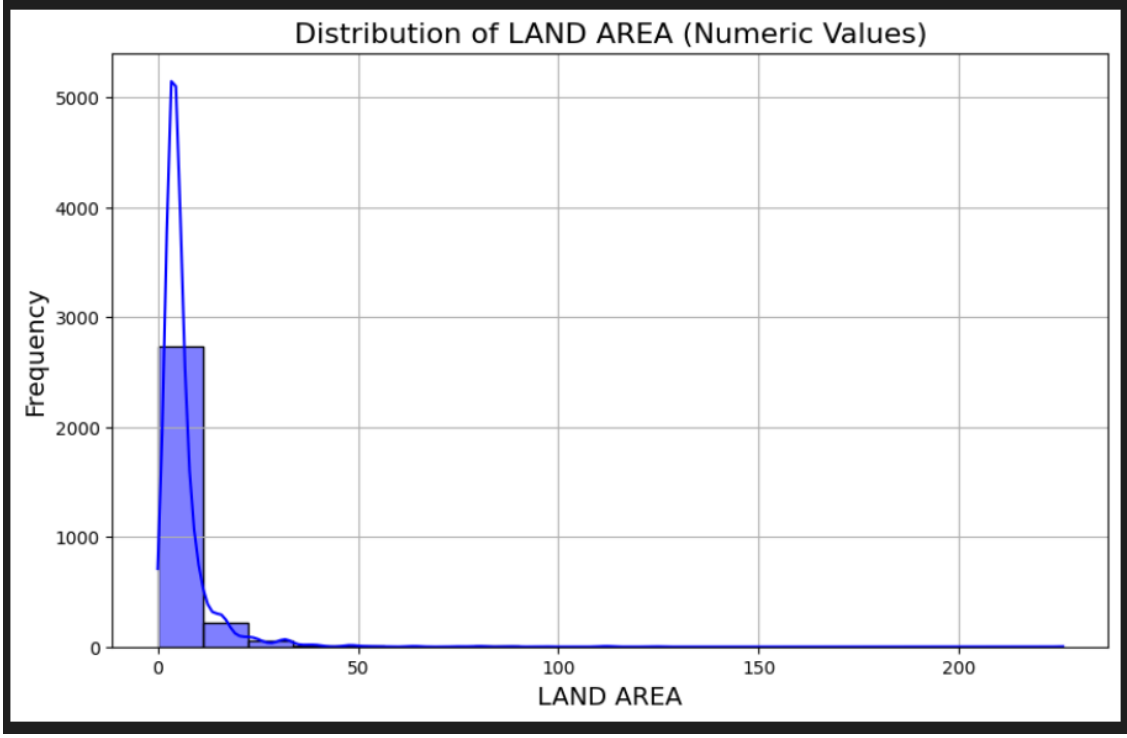


Figure 19:checking distribution of land area

```python
numeric_land_area = pd.to_numeric(df['LAND AREA'], errors='coerce')  # Convert to numeric, coercing e
median_value = numeric_land_area.median()

print(f"Median value for LAND AREA: {median_value}")

# Impute NaN values with the median
df['LAND AREA'] = df['LAND AREA'].fillna(median_value)
```

Python

Figure 20: Apply median imputation

Standard Scaling

```python
scaler1= StandardScaler()
scaler2= StandardScaler()
# Fit and transform the data
X_train_scaled= scaler1.fit_transform(X_train)
X_test_scaled= scaler1.transform(X_test)

y_train_scaled= scaler2.fit_transform(y_train.values.reshape(-1,1))
y_test_scaled= scaler2.transform(y_test.values.reshape(-1,1))
```

✓ 0.0s

Figure 21: Applying standard scaler

4.3 Model Training

4.3.1 Linear Regression

Train Test split

```python
X_train, X_test, y_train, y_test= train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ 0.0s

Figure 22: train test split

Linaer regression

```
lr= LinearRegression()
lr.fit(X_train_scaled, y_train_scaled)
```

✓ 0.0s

```
LinearRegression()
```

```
y_pred= lr.predict(X_test_scaled)
```

✓ 0.0s

Figure 23: training and predicting with linear regression model

4.3.2 Artificial Neural Network

Artificial Neural Network

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),  # First hidden layer
    Dense(32, activation='relu'),                                          # Second hidden layer
    Dense(16, activation='relu'),                                          # Third hidden layer
    Dense(1)                                                               # Output layer
])
```

✓ 0.0s

```
/home/ronak/miniconda3/envs/bajra_venv/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarnin
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='mse',  # Mean Squared Error as loss function
    metrics=[ 'mae', 'mse']
)
```

✓ 0.0s

Figure 24: ANN model building and compiling

```
# Summary of the model
model.summary()
```

✓ 0.0s Python

Model: "sequential_10"

Layer (type)	Output Shape	Param #
dense_31 (Dense)	(None, 64)	1,088
dense_32 (Dense)	(None, 32)	2,080
dense_33 (Dense)	(None, 16)	528
dense_34 (Dense)	(None, 1)	17

Figure 25: ANN model summary

```
# Train the model
history = model.fit(X_train_scaled, y_train_scaled, epochs=500, verbose=2)
```

✓ 2m 9.9s Pyth

```
Epoch 1/500
74/74 - 2s - 30ms/step - loss: 0.9000 - mae: 0.1247 - mse: 0.9000
Epoch 2/500
74/74 - 0s - 4ms/step - loss: 0.7517 - mae: 0.0975 - mse: 0.7517
Epoch 3/500
74/74 - 0s - 4ms/step - loss: 0.6341 - mae: 0.0808 - mse: 0.6341
Epoch 4/500
74/74 - 0s - 4ms/step - loss: 0.5627 - mae: 0.0779 - mse: 0.5627
Epoch 5/500
74/74 - 0s - 4ms/step - loss: 0.4839 - mae: 0.1005 - mse: 0.4839
Epoch 6/500
74/74 - 0s - 4ms/step - loss: 0.4261 - mae: 0.0901 - mse: 0.4261
Epoch 7/500
74/74 - 0s - 4ms/step - loss: 0.3058 - mae: 0.0954 - mse: 0.3058
Epoch 8/500
74/74 - 0s - 4ms/step - loss: 0.3871 - mae: 0.0935 - mse: 0.3871
Epoch 9/500
74/74 - 0s - 4ms/step - loss: 0.2018 - mae: 0.0730 - mse: 0.2018
Epoch 10/500
74/74 - 0s - 4ms/step - loss: 0.1128 - mae: 0.0631 - mse: 0.1128
Epoch 11/500
```

Figure 26: ANN training

4.4 Model Evaluation

4.4.1 Model Evaluation of Linear Regression

```
r2= r2_score(y_test_original, y_pred_original)
r2
```
✓ 0.0s

```
0.30500132565832594
```

```
accuracy= lr.score(X_test_scaled, y_test_scaled)
accuracy
```
✓ 0.0s

```
0.30500132565832616
```

Figure 27: Linear regression evaluation

```
# Evaluate the model
loss, mae, mse = model.evaluate(X_test_scaled, y_test_scaled, verbose=1)
print(f"Test Loss (MSE): {loss:.4f}")
print(f"Test MAE: {mae:.4f}")
print(f"Test MSE: {mse:.4f}")
```
✓ 0.3s Pyth

```
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0322 - mae: 0.0508 - mse: 0.0322
Test Loss (MSE): 0.0445
Test MAE: 0.0574
Test MSE: 0.0445
```

```
y_pred_scaled = model.predict(X_test_scaled)
# Inverse transform the scaled predictions and targets

y_pred_original = scaler2.inverse_transform(y_pred_scaled)
y_test_original = scaler2.inverse_transform(y_test_scaled)
```
✓ 0.2s Pyth

Figure 28: ANN evaluation

4.4.2 Model Evaluation of Artificial Neural Network

```
y_pred_scaled = model.predict(X_test_scaled)
# Inverse transform the scaled predictions and targets

y_pred_original = scaler2.inverse_transform(y_pred_scaled)
y_test_original = scaler2.inverse_transform(y_test_scaled)
```

```
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
```

```
r2 = r2_score(y_test_original, y_pred_original)
r2
```

```
0.799307956311327
```

Figure 29: ANN r2 score

5. Conclusion

5.1 Work Done Analysis

This milestone's work is on the development and training of machine learning models for the prediction of housing prices within the Nepali market. Linear Regression and ANN were chosen as the base models. Linear Regression was chosen due to its simplest model form and explainability, whereas ANN is applied to identify nonlinear patterns in the relationship. As EDA and feature engineering have not been performed, it lays a baseline for analysis of property features and their influence on the price prediction capability.

5.2 Application of the Model in Addressing Real-World Problems

Housing price prediction is crucial to the buyer, seller, and investor of real estate. This work is expected to ease decision-making processes through the provision of an efficient price estimate that will be based on critical property features such as location, land area, and road access.

The most important use cases will involve the ability of stakeholders to generalize a prediction based on a subset of features when all information is not available. In scenarios where few important features is given, the model can still approximates house prices and thereby help customers plan with limited data.

5.3 Further Improvements

The following areas are identified for further work in subsequent milestones:

a. Exploratory Data Analysis (EDA):

Detailed exploration of the dataset to uncover trends, relationships, and anomalies in the data.

b. Feature Engineering:

Transforming raw features into suitable formats and identifying the most impactful features to improve model performance.

c. Incorporating Advanced Models:

Leveraging ensemble learning techniques, such as Random Forest and Gradient Boosting, to enhance accuracy and robustness.

d. Hyperparameter Tuning:

Optimizing model parameters using techniques like GridSearchCV to improve performance.

e. Inferencing with Streamlit:

Streamlit can be used for More visual understand and interactive model inferencing.